

# OOP (Object-Oriented Programming)

Questo report ha come obiettivo di riassumere dei concetti per la programmazione in java mostrando esempi e concetti teorici.

Il primo argomento che tratto è OOP (Object-Oriented Programming).

L'OOP é la programmazione ad oggetti dove possiamo usare degli oggetti come nella vita reale. Per creare una classe (Da dove derivano gli oggetti) bisogna scrivere le parole chiavi `public class` per mettere vicino il nome della classe (deve essere con l'iniziale maiuscola), dopo queste parole mettiamo le parentesi graffe (`{}`) che racchiuderanno tutti gli attributi e metodi della classe.

Esempio:

```
public class Name_class{...}
```

Ci sono 4 pilastri della OOP

- Incapsulamento
- Ereditarietà
- Polimorfismo
- Astrazione

## INCAPSULAMENTO

Ci sono 2 elementi fondamentali nelle classi e sono **attributi** e **metodi**:

- Attributi: variabili che danno delle caratteristiche alla classe. Per crearne uno si mette un modificatore di accesso(`public` o `private` o `protected` o `package`) tipo variabile (come `int`, `String` etc...) successivamente mettere nome dell'attributo poi mettere il `;` alla fine. Di solito gli attributi si mettono `private` per non far usare all'utente gli attributi stessi ma per ricavare il suo valore ci sono i metodi `getter` e `setter`. Per chiamare l'attributo in particolare si mette **this**.

Esempio:

```
1  public class Dog{
2      private String name;
3      private int age;
4  }
```

- Metodi: Sono funzioni che appartengono a una classe. Si mette un modificatore di accesso, tipo di ritorno, nome del metodo, i parametri del metodo e poi mettere le parentesi graffe.

Esempio:

```

1  public class Dog{
2      private String  name;
3      private int     age;
4
5      public String getName() {
6          return this.name;
7      }
8
9      public int getAge() {
10         return this.age;
11     }
12
13     public void setName(String name) {
14         this.name = name;
15     }
16
17     public void setAge(int age) {
18         this.age = age;
19     }
20 }

```

|  Dog        |
|------------------------------------------------------------------------------------------------|
| - name: String<br>- age: int                                                                   |
| + getName():String<br>+ getAge():int<br>+ setName(name: String):void<br>+ setAge(age:int):void |

C'è il **Costruttore** dove inizializza gli attributi anche passati da parametri nella creazione dell'oggetto.

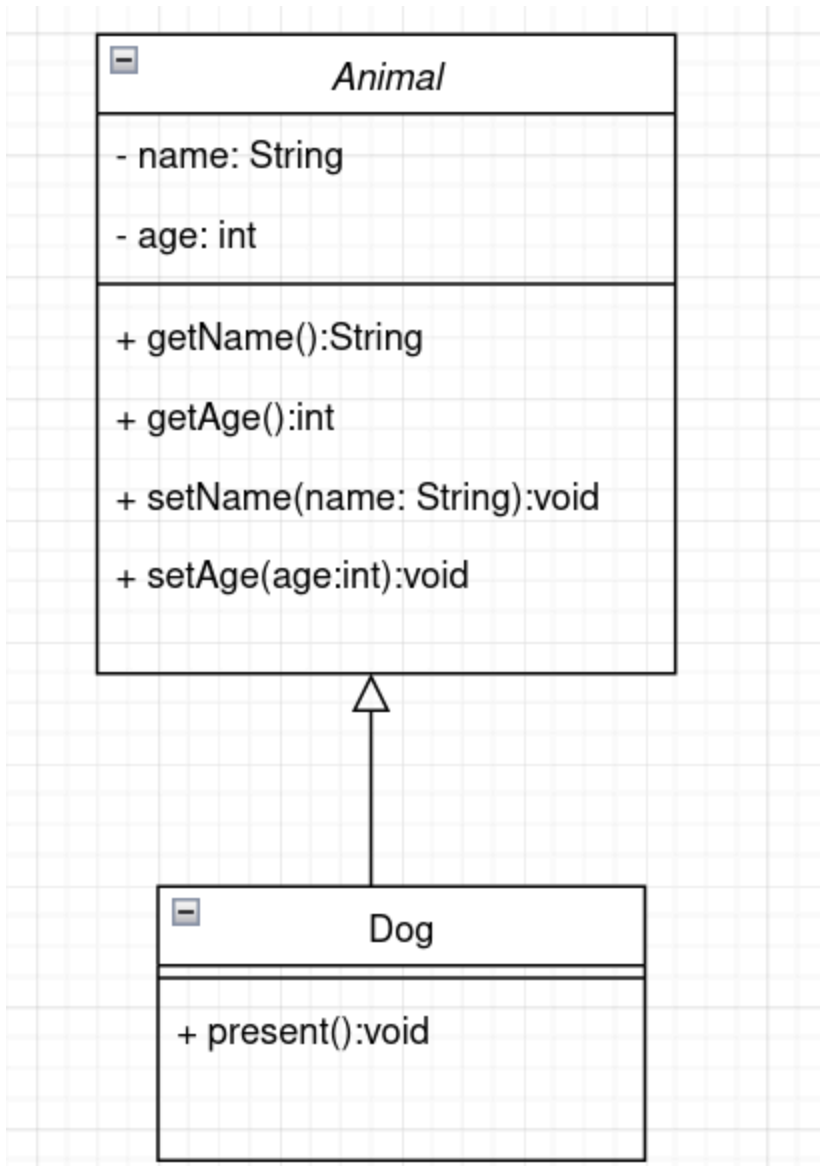
## **EREDITARIETÁ**

Un'altra classe che eredita attributi e metodi della classe padre per svolgere compiti piú precisi. Si mette **extends** vicino al nome della classe figlio per poi mettere il nome della classe padre per ereditare.

Esempio:

```
1  public class Animal{
2      private String name;
3      private int age;
4
5      public String getName() {
6          return this.name;
7      }
8
9      public int getAge() {
10         return this.age;
11     }
12
13     public void setName(String name) {
14         this.name = name;
15     }
16
17     public void setAge(int age) {
18         this.age = age;
19     }
20 }
```

```
21
22 public class Dog extends Animal{
23     public Dog(String name, int age) {
24         setName(name);
25         setAge(age);
26     }
27
28     public void present() {
29         System.out.println("I am " + this.name + " and I " + this.age + " years old");
30     }
31 }
32
33
```



## POLIMORFISMO

Il polimorfismo è quando lo stesso metodo fa cose diverse a seconda dell'oggetto che lo usa. Per usarla si deve mettere `@Override`.

Esempio:

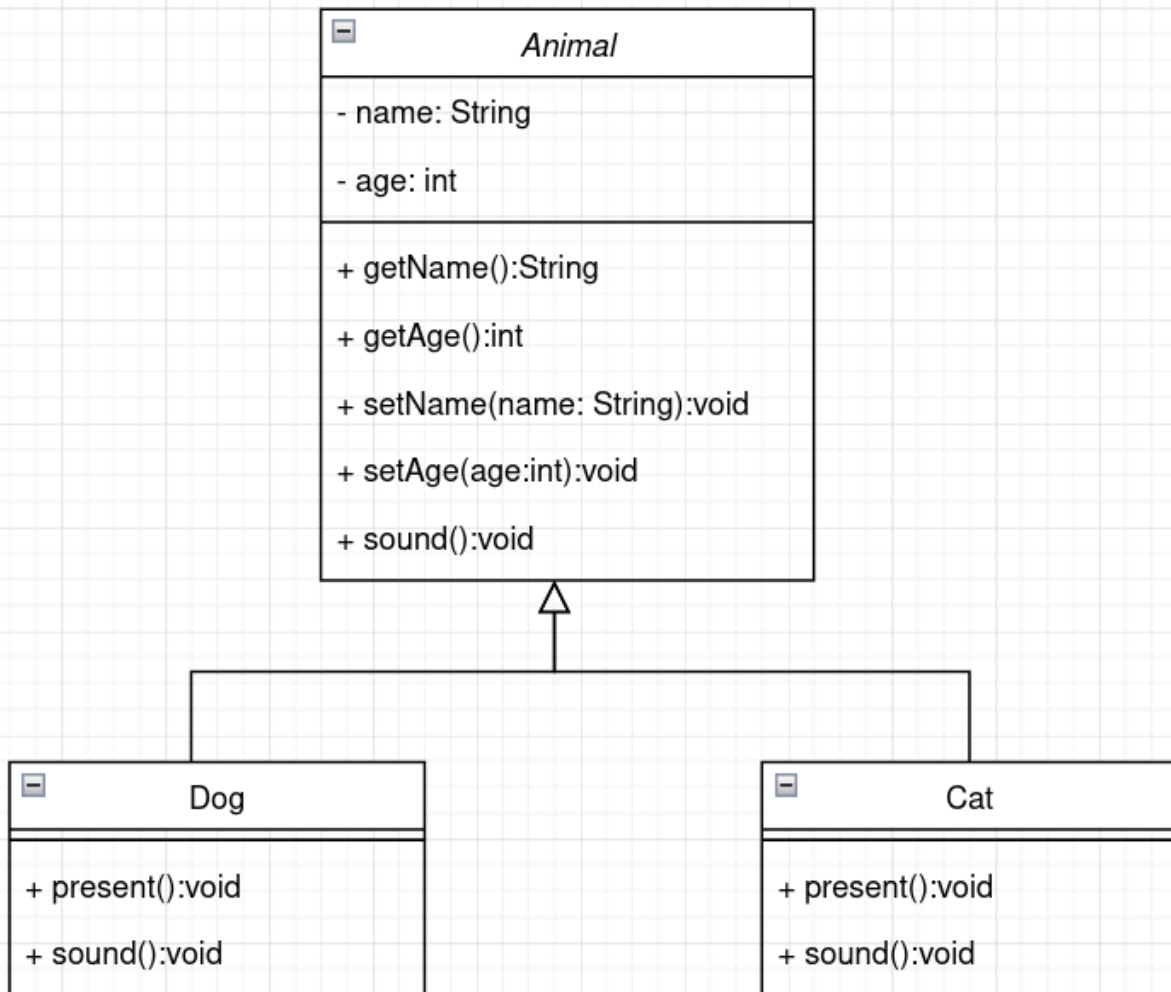
```
1  public class Animal{
2      private String name;
3      private int age;
4
5      public String getName() {
6          return this.name;
7      }
8
9      public int getAge() {
10         return this.age;
11     }
12
13     public void setName(String name) {
14         this.name = name;
15     }
16
17     public void setAge(int age) {
18         this.age = age;
19     }
20
21     public void sound() {
22         System.out.println("This method is used to hear the sounds of animals");
23     }
24 }
```

```
26 public class Dog extends Animal{
27     public Dog(String name, int age) {
28         setName(name);
29         setAge(age);
30     }
31
32     public void present() {
33         System.out.println("I am " + this.name + " and I " + this.age + " years old");
34     }
35
36     @Override
37     public void sound() {
38         System.out.println("Bau");
39     }
40 }
41
```

```

42 public class Cat extends Animal{
43     public Cat(String name, int age) {
44         setName(name);
45         setAge(age);
46     }
47
48     public void present() {
49         System.out.println("I am " + this.name + " and I " + this.age + " years old");
50     }
51
52     @Override
53     public void sound() {
54         System.out.println("Miao");
55     }
56 }

```



## ASTRAZIONE

Serve a nascondere i dettagli della classe padre, così non ci sono informazioni inutili per quell'oggetto.

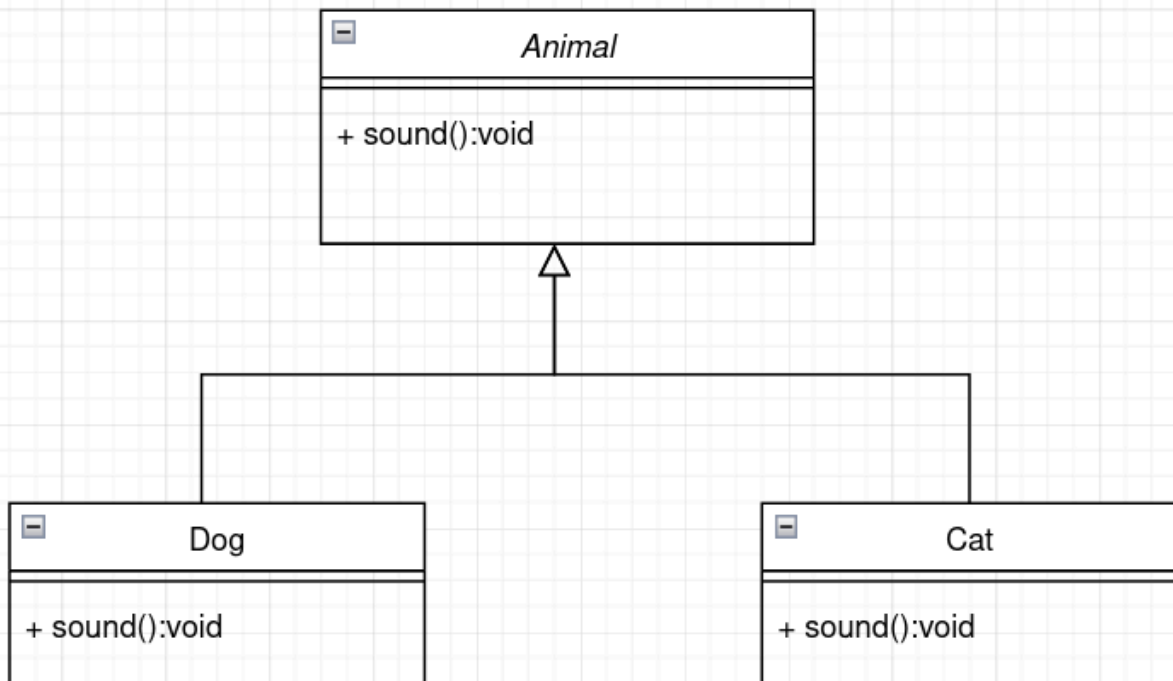
-

Esempio:

```
public interface Animal{  
    abstract void sound();  
}
```

```
public class Dog implements Animal{  
    @Override  
    void sound() {  
        System.out.println("Bau");  
    }  
}
```

```
public class Cat implements Animal{  
    @Override  
    void sound() {  
        System.out.println("Miao");  
    }  
}
```



## JUNIT



È una libreria per testare il codice, Si crea una funzione o un metodo che dati magari certi parametri bisogna sapere che risultato uscirà fuori mettendo cosa ci aspettiamo dai parametri messi. Sopra al metodo del test si deve mettere `@Test`.

Esempio:

```
9  public class DogTest{
10     @Test
11     void soundCorrect() {
12         Dog oDog = new Dog();
13         assertEquals("Bau");
14     }
15 }

26
27 public class CatTest{
28     @Test
29     void soundCorrect() {
30         Cat oCat = new Cat();
31         assertEquals("Miao");
32     }
33 }
```

## WEB PROGRAMMING

È lo sviluppo di un sito web visualizzabile solo con browser, il sito si divide in:

- **Front-end:** Parte visiva dell'utente e si usa tipo HTML per la parte grafica.
- **Back-end:** Parte invisibile per l'utente dove ci sono i meccanismi dietro alla parte del Front-end.

**MVC** (Model-View-Controller) è un pattern architetturale per le applicazioni web e si divide in 3 parti:

- **Model (JPA e EJB):** recupera dati dal database. Rappresenta:
  - logica
  - dati
  - database tramite JPA (Jakarta Persistence API)
- **View (JSP):** Visualizzazione dell'utente della parte grafica:
  - HTML
  - JSP (JavaServer Pages)
- **Controller (Servlet):** riceve richieste dall'utente e coordina il model e la view:
  - Prende l'input del browser
  - Chiama il Model
  - Ritorna una View

JSF: MVC completo: Fa insieme UI + logica.

Passaggi la web application:

- Aprire browser
- Scrivere URL
- Server Java riceve la richiesta
- Elabora i dati
- Risponde con una pagina o dati

Esempio di Login:

- Utente scrive email e password e fa login
- Server riceve email e password
- Cerca l'utente nel database, controlla la password, restituisce il risultato
- Se è andata a buon fine mostra l'homepage, se c'è un errore mostra "login fallito"